

DivideAi

Dividir despesas em grupo de forma justa, simples e transparente

Emilly Neves & David Marques

Projeto Final da disciplina · 2025/2 · Licença MIT

`github.com/DavidMarquesss/Divideai`

`Android · Kotlin 2.0.21 · minSdk 26 · Firebase (BoM 33.1.0) · Material 3`



Propósito: que problema resolvemos?

👉 **O ponto:** não é só dividir a conta — o app **simplifica as dívidas** ao mínimo de transferências.

- **Dor:** dividir contas em grupo (viagem, república, família) vira bagunça — "quem pagou o quê" e "quem deve a quem" → **atrito**
- **Solução:** registrar cada despesa (categoria, pagador, participantes, comprovante) e **dividir automaticamente**
- Saldos sempre claros: **a pagar** e **a receber**
- **Público-alvo:** amigos de viagem, colegas de república, famílias — **qualquer grupo que rateia custos** e quer transparência sem planilha

"Registrar despesas, calcular a divisão e mostrar, com o menor número de transferências, exatamente quem paga quanto a quem."

Modelo de domínio (o que é persistido)

👉 **Olhe:** `payerId` (quem pagou) + `participants` (quem deve quanto). **Por quê:** é a entrada do algoritmo e dos testes.

```
// data/model/Expense.kt
data class ExpenseShare(
    val userId: String = "",
    val amountOwed: Double = 0.0, // quanto este usuário deve ao pagador
    val paid: Boolean = false // já quitado?
)

data class Expense(
    val id: String = "", val groupId: String = "", val title: String = "",
    val amount: Double = 0.0, val date: String = "",
    val payerId: String = "", // ★ quem pagou
    val participants: List<ExpenseShare> = emptyList(), // ★ entre quem dividir
    val category: String = "", val receipt: String = "" // receipt = Base64
)
```

Arquitetura: camadas + MVVM

👉 **Olhe:** a UI só **observa** o `loginState` ; a regra pesada fica fora da tela. **Por quê:** isso a torna testável.

- `data/` = `model` · `repository` (Firestore) · `balance` (**DebtSimplifier**) · `image` (Base64) · `invite` (QR) · `notifications` (FCM); `ui/` **por feature · 81** `.kt`

```
// ui/auth/LoginViewModel.kt – estado da tela exposto como LiveData
class LoginViewModel(app: Application) : AndroidViewModel(app) {
    private val repository = AuthRepository()
    private val _loginState = MutableLiveData<LoginState>()
    val loginState: LiveData<LoginState> = _loginState // ★ só-leitura p/ a UI

    fun login(email: String, password: String) = viewModelScope.launch {
        _loginState.value = LoginState.Loading
        try { repository.signIn(email, password); _loginState.value = LoginState.Success }
        catch (e: Exception) { _loginState.value = LoginState.Error(/* msg traduzida */) }
    }
}
// LoginActivity: viewModel.loginState.observe(this) { render(it) }
```

★ Algoritmo estrela: DebtSimplifier (Kotlin puro)

👉 Olhe: o loop pega sempre o **maior credor × maior devedor** (★). Por quê: é o que produz o mínimo de transferências.

```
// 1) Saldo líquido de cada usuário
private fun computeNetBalances(expenses: List<Expense>): MutableMap<String, Double> {
    val balance = mutableMapOf<String, Double>()
    for (expense in expenses) for (share in expense.participants) {
        if (share.paid || share.userId == expense.payerId) continue
        balance.merge(expense.payerId, share.amountOwed, Double::plus) // credor +
        balance.merge(share.userId, -share.amountOwed, Double::plus) // devedor -
    }
    return balance
}
```

```
// 2) Loop guloso: maior credor × maior devedor, até zerar
while (true) {
    val creditor = balance.maxByOrNull { it.value } ?: break // ★ maior credor
    val debtor = balance.minByOrNull { it.value } ?: break // ★ maior devedor
    if (creditor.value <= EPSILON || debtor.value >= -EPSILON) break
    val amount = min(creditor.value, -debtor.value)
    transfers += SettlementTransfer(debtor.key, creditor.key, amount)
    balance[creditor.key] = creditor.value - amount
    balance[debtor.key] = debtor.value + amount // ... remove quem zerou
}
```

Build ⚙️ — Gradle Wrapper + Kotlin DSL

👉 **Por que importa:** o Wrapper fixa a versão do Gradle no repo (★) → `git clone` + `./gradlew` compila em qualquer máquina, sem instalar nada.

- **Gradle** com **Kotlin DSL** (`.kts`) — mesma linguagem do app, com autocomplete/tipos
- `gradle.properties` : `jvmargs=-Xmx2048m` · `android.useAndroidX=true` · `android.nonTransitiveRClass=true`

```
# gradle/wrapper/gradle-wrapper.properties
distributionUrl=https\://services.gradle.org/distributions/gradle-8.11.1-bin.zip # ★
```

```
// build.gradle.kts (raiz) – plugins declarados 1x, aplicados nos módulos
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    id("com.google.gms.google-services") version "4.4.4" apply false
}
```

Build — repositórios (settings.gradle.kts)

👉 **Olhe:** as origens das dependências ficam num só lugar; o **JitPack** (★) entra só por causa do MPAndroidChart.

```
// settings.gradle.kts
pluginManagement {
    repositories { google(); mavenCentral(); gradlePluginPortal() }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS) // sem repo solto nos módulos
    repositories {
        google()
        mavenCentral()
        maven { url = uri("https://jitpack.io") } // ★ p/ MPAndroidChart
    }
}
rootProject.name = "DivideAi"
include(":app")
```

Build ⚙️ — Version Catalog (`libs.versions.toml`)

👉 **Por que importa:** uma **fonte de verdade** — subir uma versão = mudar **uma linha** em `[versions]` (★).

```
# gradle/libs.versions.toml
[versions]
agp = "8.10.1"
kotlin = "2.0.21"           # ★ muda aqui → vale p/ todo o projeto
coreKtx = "1.17.0"
junit = "4.13.2"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
firebase-firestore = { module = "com.google.firebase:firebase-firestore" }
junit = { group = "junit", name = "junit", version.ref = "junit" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
```


Build ⚙️ — dependências do módulo + Firebase BoM

👉 **Olhe:** o `platform(firebase-bom)` (★) faz `auth` e `messaging` entrarem **sem número de versão**.

```
// app/build.gradle.kts
android {
    namespace = "com.example.divideai"; compileSdk = 36
    defaultConfig {
        applicationId = "com.example.divideai"; minSdk = 26; targetSdk = 36
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }
    buildFeatures { viewBinding = true } // habilita o ViewBinding
}
dependencies {
    implementation(libs.androidx.core.ktx) // alias do catálogo
    implementation(libs.material) // Material 3
    implementation(platform("com.google.firebase:firebase-bom:33.1.0")) // ★ alinha versões
    implementation("com.google.firebase:firebase-auth-ktx") // sem versão (BoM)
    implementation("com.google.firebase:firebase-messaging-ktx") // sem versão
    implementation("com.github.PhilJay:MPAndroidChart:v3.1.0") // gráficos
    implementation("com.journeyapps:zxing-android-embedded:4.3.0") // QR Code
    testImplementation(libs.junit) // JUnit 4 (testes)
}
```

Frameworks — Firebase Authentication

👉 Olhe: o `signIn` vira `suspend` (★). Por quê: o ViewModel usa `try/catch`, sem callback aninhado.

```
// data/repository/AuthRepository.kt
class AuthRepository {
    private val auth = FirebaseAuth.getInstance()

    suspend fun signIn(email: String, password: String): Result<AuthResult> = // ★
        suspendCancellableCoroutine { cont ->
            auth.signInWithEmailAndPassword(email, password)
                .addOnCompleteListener { task ->
                    if (task.isSuccessful) cont.resume(Result.success(task.result!!))
                    else cont.resumeWithException(task.exception!!)
                }
        }

    fun getCurrentUser(): FirebaseUser? = auth.currentUser // já tem alguém logado?
    fun logout() = auth.signOut()
}
```

Frameworks — Cloud Firestore (banco NoSQL)

👉 **Olhe:** `.set(...)` grava e `toObject(...)` (★) lê como objeto Kotlin. **Por quê:** tudo assíncrono, com callbacks.

```
// data/repository/ExpenseRepository.kt
class ExpenseRepository {
    private val db = Firebase.firestore
    private val expenses = db.collection("expenses")

    fun addExpense(e: Expense, onComplete: (Boolean, String?) -> Unit) {
        val doc = expenses.document()
        doc.set(e.copy(id = doc.id)) // grava (assíncrono)
        .addOnSuccessListener { onComplete(true, null) }
        .addOnFailureListener { onComplete(false, it.message) }
    }

    fun getExpensesByGroup(groupId: String, onResult: (List<Expense>) -> Unit) {
        expenses.whereEqualTo("groupId", groupId).get()
        .addOnSuccessListener { r ->
            onResult(r.documents.mapNotNull { it.toObject(Expense::class.java) }) // ★
        }.addOnFailureListener { onResult(emptyList()) }
    } // deleteExpenses(...) usa db.batch() – remoção atômica em lote
}
```

Frameworks 🔌 — Cloud Messaging (FCM / push)

👉 **Olhe:** o `token` do aparelho é salvo em `users/{uid}.fcmToken` (★). **Por quê:** é assim que o servidor sabe pra onde mandar o push.

```
// MainActivity – registra o token FCM deste aparelho no Firestore
FirebaseMessaging.getInstance().token.addOnSuccessListener { token ->
    FirebaseFirestore.getInstance()
        .collection("users").document(uid).update("fcmToken", token)    // ★
}
```

```
// notifications/DivideAiMessagingService.kt – recebe o push
class DivideAiMessagingService : FirebaseMessagingService() {
    override fun onNewToken(token: String) = saveTokenToFirestore(token)

    override fun onMessageReceived(message: RemoteMessage) {
        val title = message.notification?.title ?: message.data["title"] ?: defaultTitle
        val body = message.notification?.body ?: message.data["body"].orEmpty()
        showNotification(title, body)          // NotificationCompat.Builder(...)
    }
}
```

Frameworks — Navigation + ViewBinding + Material 3

👉 Olhe: `um` `setupWithNavController` (★) liga a barra ao grafo XML. **Por quê:** ViewBinding = acesso tipado, sem `findViewById`.

```
// MainActivity – ViewBinding + BottomNavigation ligada ao Navigation Component
binding = ActivityMainBinding.inflate(layoutInflater) // ViewBinding (sem findViewById)
setContentView(binding.root)
val navHost = supportFragmentManager
    .findFragmentById(R.id.nav_host_fragment_activity_main) as NavHostFragment
navView.setupWithNavController(navHost.navController) // ★ navView = BottomNavigationView
```

```
<!-- res/navigation/mobile_navigation.xml – grafo de navegação -->
<navigation app:startDestination="@+id/navigation_groups">
    <fragment android:id="@+id/navigation_groups"
        android:name="com.example.divideai.ui.groups.GroupsFragment" />
    <fragment android:id="@+id/navigation_expenses"
        android:name="....ui.expenses.myexpenses.MyExpensesFragment" />
    <fragment android:id="@+id/navigation_receivables" ... />
</navigation>
```

Frameworks — MPAndroidChart (dashboard)

👉 **Olhe:** dados do ViewModel viram `PieEntry` → `PieData` (★). **Por quê:** render e animação são da lib, não escrevemos gráfico na mão.

```
// ui/dashboard/DashboardActivity.kt – gráfico de pizza por categoria
private fun renderPie(state: DashboardState) {
    val entries = state.breakdown.map {
        PieEntry(it.total.toFloat(), getString(it.category.labelRes)) // ★ dado → fatia
    }
    val dataSet = PieDataSet(entries, "").apply {
        colors = state.breakdown.map { it.colorArgb }
        valueTextSize = 12f
        valueFormatter = PercentFormatter(binding.pieChart) // mostra %
    }
    binding.pieChart.data = PieData(dataSet)
    binding.pieChart.animateY(800, Easing.EaseInOutCubic) // animação
    binding.pieChart.invalidate()
}
```

Frameworks — ZXing (QR Code de convite)

👉 **Olhe:** a mesma URI `divideai://group/<id>` é **gerada** e **lida** (★). **Por quê:** a Activity Result API entrega o resultado do scanner.

```
// Gerar o QR do convite (GroupDetailsActivity)
val bitmap = BarcodeEncoder().encodeBitmap(
    GroupInviteCode.encode(groupId),    // ★ "divideai://group/<id>"
    BarcodeFormat.QR_CODE, 640, 640
)
dialogBinding.ivQrCode.setImageBitmap(bitmap)
```

```
// Ler o QR e entrar no grupo (GroupsFragment) – via Activity Result API
private val scanQrLauncher = registerForActivityResult(ScanContract()) { result ->
    val groupId = GroupInviteCode.decode(result?.contents)    // ★ mesma URI, decodificada
    ?: return@registerForActivityResult
    joinGroupFromInvite(groupId)
}
// scanQrLauncher.launch(ScanOptions().apply { setDesiredBarcodeFormats(QR_CODE) })
```

Frameworks 🔌 — Photo Picker + Base64 + ExifInterface

👉 **Por que importa:** a imagem vira **Base64** (★) e cabe no documento do Firestore → dispensa o Firebase Storage.

```
// Escolher imagem sem permissão de Storage (Jetpack Activity Result / Photo Picker)
private val pickReceiptLauncher = registerForActivityResult(
    ActivityResultContracts.PickVisualMedia()
) { uri -> if (uri != null) handlePickedReceipt(uri) }

pickReceiptLauncher.launch(
    PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.ImageOnly))
```

```
// data/image/Base64Image.kt – reduz, corrige EXIF e vira Base64 (simplificado)
fun encodeFromUri(context, uri, maxSize = SIZE_AVATAR, quality = 70): String? {
    val scaled = decodeScaled(context, uri, maxSize) // reduz resolução
    val rotated = applyExifOrientation(context, uri, scaled) // corrige orientação
    val baos = ByteArrayOutputStream()
    rotated.compress(Bitmap.CompressFormat.JPEG, quality, baos)
    return Base64.encodeToString(baos.toByteArray(), Base64.NO_WRAP) // ★ vira Base64
}
```


Frameworks 🔌 — Per-App Language (i18n pt-BR / en)

👉 Olhe: `setApplicationLocales` (★) troca o idioma **dentro do app** e persiste, sem mexer no sistema.

```
// ui/profile/ProfileActivity.kt
val locales = if (tag.isEmpty()) LocaleListCompat.getEmptyLocaleList() // segue o sistema
               else LocaleListCompat.forLanguageTags(tag)              // "pt-BR" ou "en"
AppCompatActivity.setApplicationLocales(locales) // ★ aplica e persiste entre execuções
```

- Idiomas declarados em `res/xml/locales_config.xml` ; strings em `values/` (pt) e `values-en/`

Testes 📱 — estrutura e configuração

👉 **Por que importa:** `DebtSimplifier` é Kotlin puro → testa na **JVM, sem emulador** (★ `testImplementation`).

- `app/src/test/...` roda **sem device** • `app/src/androidTest/...` precisa emulador
- Comandos: `./gradlew test` • `./gradlew connectedAndroidTest`
- Alvo escolhido: **`DebtSimplifier`** → **5 casos, todos passando** ✅

```
// app/build.gradle.kts
defaultConfig { testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner" }
dependencies {
    testImplementation(libs.junit)                // ★ JUnit 4.13.2 (unitário, JVM)
    androidTestImplementation(libs.androidx.junit) // AndroidX Test
    androidTestImplementation(libs.androidx.espresso.core) // Espresso (UI)
}
```

Testes 📱 — DebtSimplifierTest (JUnit)

👉 Olhe: o `assertEquals` prova que $A \rightarrow B \rightarrow C$ vira **1** transferência $A \rightarrow C$ (★) — o intermediário some.

```
private fun debt(payer: String, debtor: String, amount: Double) =
    Expense(payerId = payer, participants = listOf(ExpenseShare(debtor, amount)))

@Test fun `cadeia A deve B e B deve C vira transferencia A para C`() {
    val transfers = DebtSimplifier.simplify(listOf(
        debt("B", "A", 10.0), // A deve 10 a B
        debt("C", "B", 10.0))) // B deve 10 a C
    assertEquals(1, transfers.size) // uma única transferência
    assertEquals("A", transfers[0].debtorId)
    assertEquals("C", transfers[0].creditorId) // ★ o intermediário B some
    assertEquals(10.0, transfers[0].amount, 0.01)
}

@Test fun `despesa rateada gera uma transferencia de cada devedor para o pagador`() {
    val jantar = Expense(payerId = "A", participants = listOf(
        ExpenseShare("A", 10.0), ExpenseShare("B", 10.0), ExpenseShare("C", 10.0)))
    val t = DebtSimplifier.simplify(listOf(jantar))
    assertEquals(2, t.size); assertTrue(t.all { it.creditorId == "A" })
}
```

+ 3 casos: dívida direta · parcelas já pagas ignoradas · lista vazia

Opcionais — Issue tracking + CI/CD (GitHub Actions)

👉 **Por que importa:** todo push roda `./gradlew test` (★) → se o algoritmo quebrar, o CI acusa antes do merge.

- **Issue tracking: GitHub Issues** · **CI:** publica **relatório de testes** e **APK** como artefatos

```
# .github/workflows/android.yml
on:
  push: { branches: [ main ] }
  pull_request: { branches: [ main ] }
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - uses: android-actions/setup-android@v3
      - run: ./gradlew test          # ★ roda o DebtSimplifierTest
      - run: ./gradlew assembleDebug # gera o APK de debug
```

Opcionais — Containers + o extra: notifier (Node.js)

👉 **Olhe:** `onSnapshot` (★) reage a despesa nova em tempo real e notifica os participantes — de **fora** do app (Node).

- **Containers:** *não feito* — app Android não roda em container; o `notifier/` poderia ser containerizado (evolução)
- **Extra:** `notifier/` em **Node.js** com `firebase-admin` (Admin SDK) → projeto **poliglota**

```
// notifier/watch.js – escuta o Firestore em tempo real e notifica
db.collection('expenses').where('createdAt', '>=', startedAt)
  .onSnapshot((snap) => {                                     // ★ reage em tempo real
    for (const change of snap.docChanges()) {
      if (change.type !== 'added') continue;
      const data = change.doc.data();
      for (const p of (data.participants || [])) {
        if (p.userId === data.payerId) continue;             // não avisa quem pagou
        sendToUser(p.userId, 'Nova despesa no grupo', `Incluído em "${data.title}"`);
      }
    }
  });
```

Demonstração ao vivo (opcional)

👉 **O clímax:** mostrar o pulo de **Saldos** → **Saldos Simplificados** (menos transferências).

1. Login → Criar grupo ("Viagem Praia")
2. Convidar membro por **QR Code**
3. Lançar despesa: **categoria + pagador + participantes + comprovante**
4. Ver **Saldos** → **SALDOS SIMPLIFICADOS**
5. **Dashboard:** gráfico de pizza por categoria
6. *(opcional)* rodar o `notifier` → push chegando

Plano B: prints em `docs/screenshots/` (01 → 05) seguem o mesmo fluxo se o emulador falhar.

Encerramento

1. **Problema real:** dividir contas em grupo e saber "quem deve a quem" gera confusão
2. **Solução:** registrar despesas, dividir e **SIMPLIFICAR** as dívidas ao mínimo de transferências (`DebtSimplifier` , Kotlin puro, **testado**)
3. **Stack sólida:** Android nativo + Jetpack + Material 3 + Firebase; build reproduzível (Wrapper + Version Catalog); **testes + CI**

Diferenciais: testes + CI · QR Code · Base64 no Firestore · modo escuro · i18n · notifier Node.js

Próximos passos: Espresso de UI · containerizar o notifier · marcar dívida como paga

Obrigado! 🐮

Perguntas?

Emilly Neves & David Marques

Projeto Final 2025/2 · Licença MIT

github.com/DavidMarquesss/Divideai

